

Software Architecture and Design Specification

Project: Vehicle Parking System (Project No. 10) – a system for managing parking slots and vehicle tracking

Version: 1.1

Authors: Team 10 – Ramitha R, Pandi Snehitha, Pramiti Ragavendra Udupa

Date: 07-10-2026

Status: Draft – submitted for review

Course: Software Engineering (UE24CS341A) | Semester 5 | Section F | PES University, Bengaluru

Team Profile

Sl. No	Name	SRN
1	Ramitha R	PES1UG24CS368
2	Pandi Snehitha	PES1UG24CS315
3	Pramiti Ragavendra Udupa	PES1UG24CS331

Revision History

Version	Date	Author	Change Summary
1.0	05-10-2026	Pramiti Ragavendra Udupa	Initial architecture and design, prepared from the problem statement
1.1	07-10-2026	Pramiti Ragavendra Udupa	Aligned with SRS v1.1 and Software Test Plan v1.1: requirement IDs FR-01...FR-33, NFR-01...NFR-12, SEC-01...SEC-10, BR-01...BR-06; components renamed to match the SRS RTM; reservations, online payment and e-mail removed (out of scope in SRS 1.3); new sequence diagrams, API, role matrix, data model and traceability to STP test cases

Approvals

Role	Name	Signature/Date
Author (Architecture & Design)	Pramiti Ragavendra Udupa	
Reviewer (SRS)	Pandi Snehitha	
Reviewer (QA Lead, Test Plan)	Ramitha R	
Course Faculty		

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Vehicle Parking System (VPS). It turns the requirements of the VPS Software Requirements Specification (SRS v1.1) into components, interfaces, data stores, security controls and interaction flows, and records the reasons for each major architectural decision. Every requirement ID used here is the ID defined in the SRS, and every test-case ID is the ID defined in the Software Test Plan (STP v1.1).

1.2 Scope

The VPS is a web-based system that keeps an accurate, current record of which parking slots are free, which vehicle is parked in which slot, when it entered and left, and how much it owes. This document covers the architecture and design of:

- Authentication, Vehicle Owner self-registration and role-based access control (FR-01 – FR-06).
- Vehicle registration and maintenance (FR-07 – FR-10).
- Parking slot management and slot availability display (FR-11 – FR-15).
- Vehicle entry with automatic slot allocation (FR-16 – FR-19).
- Vehicle exit, parking fee calculation and slot release (FR-20 – FR-24).
- Parking fee rule configuration (FR-25 – FR-27).
- Vehicle search, current-location tracking and parking history (FR-28 – FR-31).
- Parking Occupancy and Vehicle Tracking reports (FR-32, FR-33).
- The non-functional, security and business-rule requirements NFR-01 – NFR-12, SEC-01 – SEC-10 and BR-01 – BR-06.

Out of scope (SRS 1.3): online payment, number-plate recognition, hardware gate control, reservations and native mobile applications. Fees are calculated and recorded by the system but collected outside it (cash or counter). Version 1.0 of the VPS has no interface to any other system (SRS 2.1).

1.3 Audience

The Team 10 developers, the testers who carry out the Software Test Plan, course faculty and reviewers, security reviewers, and future maintainers.

1.4 Definitions

Term	Definition
VPS	Vehicle Parking System
Vehicle Owner / Parking Attendant / Administrator	The three user roles of the SRS (2.3); written OWNER, ATTENDANT and ADMIN in the API
Parking Slot	A uniquely identified parking space (e.g. A-01) with a slot type and a status
Slot Status	Available, Occupied or Unavailable

Term	Definition
Parking Record	One visit: vehicle, slot, ticket number, entry time, exit time, fee and status (Active or Completed)
Ticket Number	Unique number given to a parking record at entry
Grace Period	Minutes at the start of a visit for which no fee is charged (0–60, set by the Administrator)
Billable Hours	Parking duration minus the grace period, rounded up to the next whole hour (0 if within the grace period)
FR / NFR / SEC / BR	Functional / Non-Functional / Security requirement / Business Rule (SRS IDs)
TC	Test case (STP IDs, e.g. TC-ENT-01)
MERN	MongoDB, Express.js, React.js, Node.js
SPA	Single Page Application (the React.js front end)
REST / API / JSON	Representational State Transfer / Application Programming Interface / JavaScript Object Notation
JWT	JSON Web Token – the signed session token
RBAC	Role-Based Access Control
ADR	Architecture Decision Record
STRIDE	Threat model: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS / HTTPS	Transport Layer Security / HTTP over TLS
ODM	Object Document Mapper (Mongoose)
SRS / STP / RTM	Software Requirements Specification / Software Test Plan / Requirements Traceability Matrix

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture: goals and constraints, stakeholders, the UML component diagram and component descriptions, the architecture pattern, the technology stack and data stores, risks, traceability to every SRS requirement, the security architecture, the deployment view and the architecture decision records. Section 4 describes the design: module structure and state models, four UML sequence diagrams, the REST API with its role matrix, error handling, logging and monitoring, UX design, and open issues. Section 5 holds the glossary, references and tools. The traceability table in Section 3.8 supplies the Architecture Reference and Design Reference columns of the RTM in SRS Appendix C.

2.2 Related Documents

- **SRS** – Vehicle Parking System Software Requirements Specification v1.1 (Team 10), including the RTM in Appendix C.
- **STP** – Vehicle Parking System Software Test Plan v1.1 (Team 10), test cases in Appendix B.
- **RTM** – Requirements Traceability Matrix (SRS Appendix C; STP Section 13).
- **Problem statement** – SE Mini-Project Problem Statements, Section F, Project No. 10: "Vehicle Parking System – a system for managing parking slots and vehicle tracking".

3. Architecture

3.1 Goals & Constraints

Goals:

- **Correct allocation:** a slot never has two active vehicles and a vehicle never has two active records, even under concurrent requests (NFR-05, BR-01, BR-03).
- **Durability:** every acknowledged entry or exit survives a server restart (NFR-06).
- **Performance:** vehicle search ≤ 3 s and slot display ≤ 2 s for 95% of requests with 50 concurrent users; entry and exit ≤ 3 s for 95% of operations (NFR-01 – NFR-04).
- **Availability:** at least 99% of configured operating hours per month (NFR-07).
- **Security:** authenticated, role-based access enforced on the server; hashed passwords; TLS; audit of failed and denied requests (SEC-01 – SEC-10).
- **Maintainability and testability:** separate authentication, vehicle, slot, parking-transaction, fee and report modules that talk only through service interfaces; $\geq 70\%$ unit-test coverage of backend services (NFR-10, NFR-11).
- **Usability and portability:** vehicle entry in ≤ 5 user actions and ≤ 60 s; works in the latest two Chrome, Firefox and Edge versions from 360 px wide (NFR-08, NFR-09, NFR-12).

Constraints:

- Must be built with the MERN stack and store all data in a persistent MongoDB database (SRS 2.5).
- No interfaces to external systems in Version 1.0; no online payment, reservations, number-plate recognition or gate hardware (SRS 1.3, 2.1).
- Timestamps recorded in server time (IST) and stored in UTC; amounts in Indian rupees with two decimal places (SRS 7).
- SRS, SAD and Test Plan must stay traceable through the RTM (SRS 2.5).
- Three-member team working within the mini-project schedule.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Vehicle Owner	Register own vehicles, see free slots, find where their vehicle is, see own history; privacy of own data (SEC-09)
Parking Attendant	Fast and simple entry and exit (≤ 5 actions), correct slot allocation and fee, clear error messages
Administrator	Manage slots, users and fee rules; accurate occupancy and vehicle tracking reports; only admins can change these (BR-05)
Developers (Team 10)	Clear module boundaries, a stack the team knows, testable services
Testers	Requirement IDs and test-case IDs that line up; a role matrix to test RBAC (STP TC-SEC-02)
Course faculty / reviewers	IEEE-style documentation, sound architecture rationale, security and traceability coverage

3.3 Component (UML) Diagram

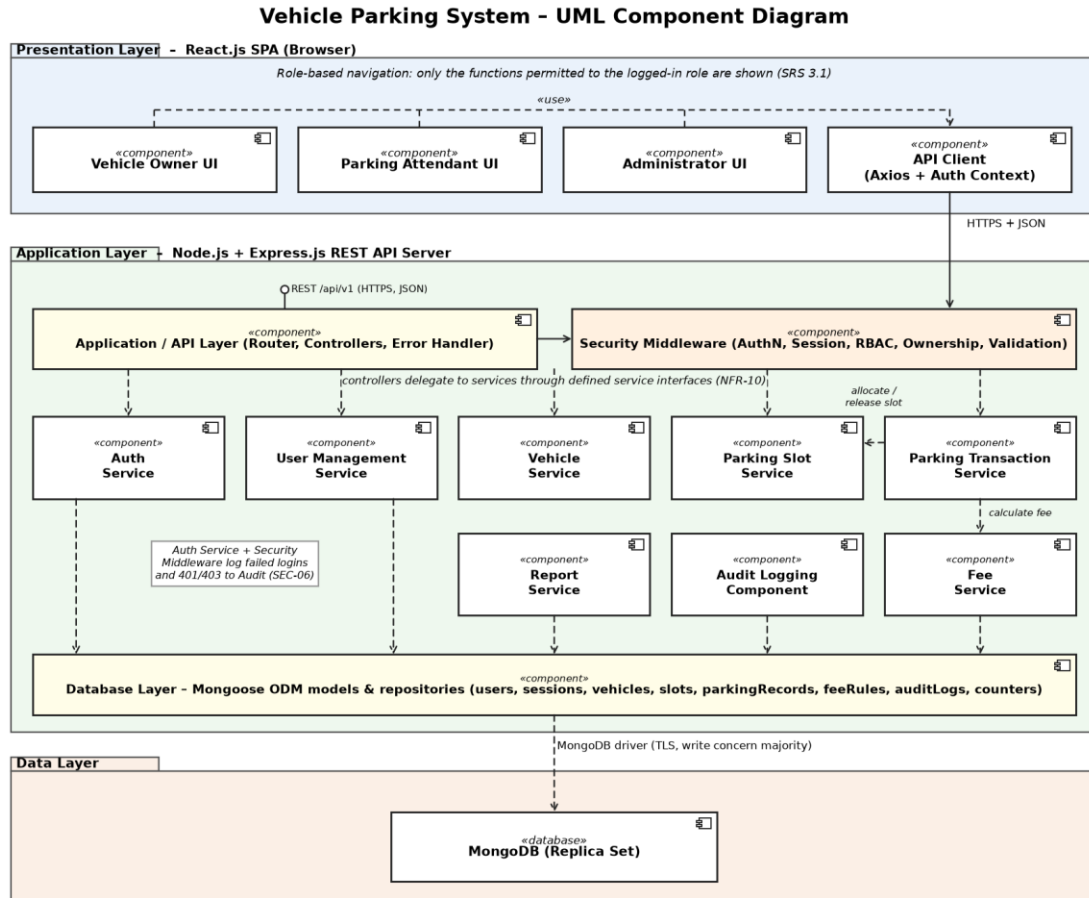


Figure 1: UML component diagram of the Vehicle Parking System (layered, MERN)

3.4 Component Descriptions

Component names match the Architecture Reference column of the SRS RTM (Appendix C).

Component	Responsibility	Requirements
Presentation Layer (React.js SPA)	Vehicle Owner UI, Parking Attendant UI and Administrator UI. Shows only the functions allowed for the logged-in role; marks mandatory fields; shows user, role and Logout on every screen. Uses the API Client (Axios) which attaches the token and maps error codes to messages.	SRS 3.1, NFR-08, NFR-09, NFR-12
Application / API Layer	Express.js router and controllers for /api/v1; controllers parse requests, call one service and shape the HTTP response; a central error handler returns the standard error format (Section 4.4).	SRS 3.2, 3.3, NFR-01 – NFR-04
Security Middleware	Runs before every controller: verifies the JWT and its server-side session (logout, 15-minute idle timeout), checks the role against the role matrix, applies owner-only data rules, validates and sanitises input, and reports 401/403 to the Audit Logging Component.	FR-06, SEC-01, SEC-02, SEC-05, SEC-07, SEC-09
Auth Service	Login, logout, Vehicle Owner self-registration, bcrypt password hashing and checking, failed-attempt counting and 15-minute account lock-out, session creation and revocation.	FR-01 – FR-04, SEC-03, SEC-07, SEC-08
User Management Service	Administrator creates, updates, deactivates and assigns roles to Parking Attendant and Administrator accounts.	FR-05, BR-05

Component	Responsibility	Requirements
Vehicle Service	Register, validate (4–15 characters, letters/digits/hyphen/space, stored upper case), update type, deactivate; search by registration number with owner restriction.	FR-07 – FR-10, FR-28, FR-29, SEC-09
Parking Slot Service	Add slots, change slot type (not when Occupied), set Unavailable/Available (not when Occupied), list slots and per-type counts, and the atomic allocate/release operations used by the Parking Transaction Service.	FR-11 – FR-15, FR-17, BR-02
Parking Transaction Service	Vehicle entry (check no active record, allocate slot, create record with ticket number), exit (store exit time, get fee, show summary), confirm exit (complete record, release slot), current status, history, list of parked vehicles.	FR-16 – FR-24, FR-29 – FR-31, NFR-05, BR-01, BR-03, BR-06
Fee Service	Stores hourly rate per vehicle type and the grace period; calculates the fee from the rule in force when the exit is recorded.	FR-22, FR-25 – FR-27, BR-04
Report Service	Parking Occupancy Report (each slot with status, assigned vehicle and last-updated time) and Vehicle Tracking Report (records of one vehicle in a date range).	FR-32, FR-33
Audit Logging Component	Writes failed logins and every 401/403 with timestamp, username and source IP (never passwords) to an append-only collection kept at least 90 days.	SEC-06
Database Layer	Mongoose schemas, validation, unique and partial-unique indexes, transactions and repositories for all collections (Section 3.6).	NFR-06, SRS 7
MongoDB (Data Layer)	Persistent store run as a 3-member replica set (needed for multi-document transactions and fail-over), journaled writes with write concern majority.	NFR-05 – NFR-07

3.5 Chosen Architecture Pattern and Rationale

Pattern: layered (three-tier client–server) architecture. Inside the back end the layers are Router/Controller → Security Middleware → Service → Database Layer (repository), and each layer calls only the layer below it.

- **Presentation layer:** React.js SPA in the browser; holds no business rules (input hints only).
- **Application layer:** Node.js/Express.js REST API containing the security middleware and the six business modules required by NFR-10 (authentication, vehicle, slot, parking transaction, fee, report) plus user management and audit logging.
- **Data layer:** MongoDB accessed only through Mongoose models in the Database Layer.

Rationale:

- NFR-10 asks for separate modules that exchange data only through defined service interfaces – a layered, modular back end gives exactly that and lets each service be unit-tested with a mocked repository (NFR-11).
- All security checks (authentication, sessions, RBAC, ownership, validation) sit in one middleware layer, so every endpoint is protected the same way (SEC-01, SEC-02, SEC-05).
- Matches the MERN stack required by the SRS and the team's skills, keeping the schedule realistic.
- Business rules that must never break (BR-01 – BR-03) are enforced in the service layer and backed by database indexes and transactions.

Alternatives considered and rejected:

- **Microservices:** rejected – one parking facility does not need independent scaling; distributed transactions would make NFR-05 (no double allocation) much harder.
- **Server-rendered monolith (templates):** rejected – the SRS asks for a React.js front end calling a REST API.
- **Real-time push (WebSockets):** not required by the SRS; the slot view refreshes on load, on every entry/exit and every 30 s, which meets NFR-02 with less complexity.

3.6 Technology Stack & Data Stores

Layer / Concern	Technology
Front end	React.js 18, Vite, React Router, Axios, Tailwind CSS
Back end	Node.js LTS, Express.js 4, Joi (validation), express-mongo-sanitize, Helmet
Database	MongoDB (3-member replica set), Mongoose ODM; separate test database
Security	bcrypt (cost factor 12, minimum 10 per SEC-03), jsonwebtoken (HS256), server-side session store, TLS 1.2+ via Nginx
Logging	Winston (JSON logs, passwords never logged), audit collection in MongoDB
Testing (per STP)	Jest, Supertest, Cypress, Postman/Newman, Apache JMeter, OWASP ZAP, testssl.sh, gitleaks, npm audit
DevOps	Git and GitHub (team repository, GitHub Issues for defects), PM2, Nginx on Ubuntu 22.04

Data stores (MongoDB collections, from SRS Section 7 and Appendix B):

Collection	Key Fields	Indexes / Notes
users	_id, username, name, email, phone, passwordHash, role (OWNER ATTENDANT ADMIN), active, failedAttempts, lockUntil	unique(username)
sessions	jti, userId, createdAt, lastActivityAt, revoked	unique(jti); TTL removes expired sessions
vehicles	_id, regNo (upper case), vehicleType (TWO_WHEELER CAR OTHER), ownerId, active	unique(regNo); index(ownerId)
slots	slotId (e.g. A-01), slotType, status (AVAILABLE OCCUPIED UNAVAILABLE), lastUpdated	unique(slotId); index(slotType, status, slotId)
parkingRecords	ticketNo, vehicleId, slotId, entryTime, exitTime, durationMin, fee, status (ACTIVE COMPLETED), recordedBy	unique(ticketNo); partial-unique(vehicleId) and partial-unique(slotId) where status = ACTIVE (BR-01, BR-03)
feeRules	vehicleType, hourlyRate (₹, ≥ 0); one feeSettings document with gracePeriodMin (0–60); updatedAt, updatedBy	unique(vehicleType)
auditLogs	timestamp, event (FAILED_LOGIN UNAUTHENTICATED FORBIDDEN ACCOUNT_LOCKED), username, sourceIp, method, path, status	index(timestamp); TTL 365 days (≥ 90 days per SEC-06); no update or delete API
counters	name, seq	Generates unique ticket numbers, e.g. T-20261007-0042

Fee rule (FR-22, BR-04): duration $d = \text{exitTime} - \text{entryTime}$ in minutes; if $d \leq \text{gracePeriod}$ the fee is ₹0, otherwise $\text{billableHours} = \text{ceil}((d - \text{gracePeriod}) \div 60)$ and $\text{fee} = \text{billableHours} \times \text{hourlyRate}$ of the vehicle type. The rule in force when the exit is recorded is used (FR-27), and a Completed record keeps its stored fee (BR-06). Money is stored as an integer number of paise and shown with two decimals. With the proposed defaults (Car ₹30/hour, grace 15 min): 15 min → ₹0, 16 min → ₹30, 76 min → ₹60, 195 min → ₹90, which matches STP TC-EXT-03.

3.7 Risks & Mitigations

#	Risk	Mitigation
R1	Two attendants allocate the same slot at the same moment (NFR-05)	Atomic findOneAndUpdate on slot status inside a transaction, plus partial-unique index on Active records per slot; the losing request gets 409 and is asked to retry (tested by TC-CONC-01)
R2	Acknowledged entry/exit lost on a crash (NFR-06)	Replica set with journaled, majority-acknowledged writes; respond only after commit
R3	Default hourly rates not yet confirmed (SRS 2.6)	Rates and grace period are data in feeRules/feeSettings, editable by the Administrator; nothing is hard-coded
R4	Brute-force or credential-stuffing on login	5-attempt lock-out for 15 minutes (SEC-08), generic error message (FR-02), audit of failures (SEC-06)
R5	Owner reads another owner's data	Ownership check in Security Middleware and Vehicle Service; non-owned vehicles reported as not found (SEC-09)
R6	Database host failure lowers availability (NFR-07)	3-member replica set with automatic fail-over; daily backups kept 7 days; PM2 restarts the API on failure
R7	SRS and Test Plan drift apart (requirement IDs differ)	This SAD uses the SRS IDs; Section 3.8 lists requirements without a test case yet so the STP can be updated

3.8 Traceability to Requirements

Each SRS requirement is mapped to the architecture component that realises it, the design section of this document, and the STP v1.1 test case(s) that verify it (matched by what the test checks). Items marked "Not yet in STP" have no test case in STP v1.1 and need one added.

Req. ID	Requirement (summary)	Architecture Reference	Design Reference	Test Case ID (STP v1.1)
FR-01	Login, issue session token	Auth Service	4.2 SD-3; 4.3 (a)	TC-AUTH-01
FR-02	Generic invalid-credentials message	Auth Service	4.2 SD-3; 4.4	TC-AUTH-02
FR-03	Logout invalidates token	Auth Service, Security Middleware	3.9; 4.3	TC-AUTH-03
FR-04	Owner self-registration	Auth Service	4.3	Not yet in STP
FR-05	Admin manages accounts and roles	User Management Service	4.3 role matrix	TC-AUTH-04
FR-06	403 for functions outside role	Security Middleware	3.9; 4.3 role matrix	TC-AUTH-05
FR-07	Register a vehicle	Vehicle Service	4.3 (b)	TC-VEH-01
FR-08	Reject empty / duplicate fields	Vehicle Service	4.3 (b); 4.4	TC-VEH-02
FR-09	Registration-number format, upper case	Vehicle Service, Security Middleware	4.3 (b)	TC-VEH-01, TC-VEH-03
FR-10	Update / deactivate vehicle if not parked	Vehicle Service	4.3	Not yet in STP
FR-11	Admin adds a slot	Parking Slot Service	4.3 (c)	TC-SLOT-01
FR-12	Change slot type if not Occupied	Parking Slot Service	4.1 state model; 4.3	Not yet in STP
FR-13	Set Unavailable (not if Occupied)	Parking Slot Service	4.1 state model; 4.3	TC-SLOT-02
FR-14	Display every slot and status	Parking Slot Service, Presentation Layer	4.3; 4.5	TC-SLOT-03
FR-15	Slot counts per type	Parking Slot Service	4.3	TC-SLOT-03
FR-16	Record entry; reject if already parked	Parking Transaction Service	4.2 SD-1; 4.3 (d)	TC-ENT-01, TC-ENT-02
FR-17	Allocate lowest matching slot / manual choice	Parking Transaction + Parking Slot Service	4.2 SD-1	TC-ENT-01, TC-ENT-04
FR-18	Refuse when no matching slot	Parking Transaction Service	4.2 SD-1 (alt)	TC-ENT-03
FR-19	Atomic record + ticket + slot Occupied	Parking Transaction Service, Database Layer	4.2 SD-1; 3.6	TC-ENT-01, TC-CONC-01
FR-20	Record exit; reject without active record	Parking Transaction Service	4.2 SD-2; 4.3 (e)	TC-EXT-01, TC-EXT-02
FR-21	Store exit time (server time)	Parking Transaction Service	4.2 SD-2	TC-EXT-01
FR-22	Fee = billable hours × hourly rate	Fee Service	3.6 fee rule; 4.2 SD-2	TC-EXT-03

Req. ID	Requirement (summary)	Architecture Reference	Design Reference	Test Case ID (STP v1.1)
FR-23	Show fee summary before confirm	Fee Service, Presentation Layer	4.2 SD-2; 4.5	TC-EXT-01
FR-24	Complete record, release slot	Parking Transaction Service	4.2 SD-2	TC-EXT-01
FR-25	Hourly rate per type (≥ 0)	Fee Service	4.3 (f)	TC-FEE-01
FR-26	Grace period 0–60 minutes	Fee Service	4.3 (f)	Not yet in STP (STP treats grace as fixed)
FR-27	Rule change applies to later exits only	Fee Service	3.6 fee rule	TC-FEE-02
FR-28	Search vehicle (owner: own only)	Vehicle Service	4.2 SD-4; 4.3 (g)	TC-SRCH-01
FR-29	Current status Parked / Not Parked	Vehicle Service, Parking Transaction Service	4.2 SD-4	TC-SRCH-02
FR-30	Parking history, newest first	Parking Transaction Service	4.2 SD-4	TC-SRCH-03
FR-31	List currently parked vehicles	Parking Transaction Service	4.3	Not yet in STP
FR-32	Parking Occupancy Report	Report Service	4.3	TC-REP-01
FR-33	Vehicle Tracking Report (date range)	Report Service	4.3	Not yet in STP
NFR-01	Search ≤ 3 s, 95%, 50 users	Application/API Layer, Database Layer	3.6 indexes; 4.4	TC-PERF-01
NFR-02	Slot display ≤ 2 s, 95%, 50 users	Application/API Layer	3.6 indexes; 4.4	Not yet in STP
NFR-03	Entry ≤ 3 s, 95%	Parking Transaction Service	4.2 SD-1; 4.4	TC-PERF-02
NFR-04	Exit ≤ 3 s, 95%	Parking Transaction Service	4.2 SD-2; 4.4	TC-PERF-02
NFR-05	No double allocation under concurrency	Parking Transaction Service, Database Layer	4.2 SD-1 (alt); 3.11 ADR-02	TC-CONC-01
NFR-06	Acknowledged operations persisted	Database Layer, MongoDB	3.10; 3.11 ADR-02	TC-REL-01
NFR-07	99% availability in operating hours	Deployment Architecture	3.10	Not yet in STP
NFR-08	Entry in ≤ 5 actions, ≤ 60 s	Presentation Layer	4.5	TC-USAB-01
NFR-09	Helpful error messages, * on mandatory fields	Presentation Layer, API Layer	4.4; 4.5	Not yet in STP
NFR-10	Separate modules with service interfaces	All services	3.5; 4.1	Not yet in STP (code review)
NFR-11	$\geq 70\%$ unit-test coverage of services	All services	4.1	Not yet in STP
NFR-12	Latest two Chrome/Firefox/Edge, ≥ 360 px	Presentation Layer	4.5	TC-PORT-01
SEC-01	Authenticate before protected functions	Security Middleware	3.9; 4.3 role matrix	TC-AUTH-01, TC-SEC-01
SEC-02	Server-side RBAC on every endpoint	Security Middleware	3.9; 4.3 role matrix	TC-AUTH-05, TC-SEC-02
SEC-03	bcrypt hashes, cost ≥ 10 ; no passwords in logs	Auth Service	3.9; 4.4	TC-SEC-03
SEC-04	HTTPS, TLS 1.2+	Deployment Architecture	3.10	TC-SEC-04
SEC-05	Server-side input validation	Security Middleware	3.9; 4.4	TC-SEC-05
SEC-06	Log failed logins and 401/403, keep 90 days	Audit Logging Component	3.6; 4.4	Not yet in STP
SEC-07	End session after 15 min inactivity	Auth Service, Security Middleware	3.9	TC-SEC-06
SEC-08	Lock account 15 min after 5 failures	Auth Service	4.2 SD-3	Not yet in STP
SEC-09	Owner reads only own vehicles' data	Security Middleware, Vehicle Service	4.2 SD-4; 4.3	Not yet in STP
SEC-10	Secrets in environment configuration	Deployment Architecture	3.10	TC-SEC-07
BR-01	One active record per slot	Parking Transaction Service, Database Layer	3.6 indexes	TC-CONC-01
BR-02	Allocate only Available, type-matching slots	Parking Slot Service	4.2 SD-1	TC-ENT-01
BR-03	One active record per vehicle; exit needs one	Parking Transaction Service	3.6 indexes; 4.2 SD-2	TC-ENT-02, TC-EXT-02
BR-04	Fees only from admin-set rules	Fee Service	3.6 fee rule	TC-EXT-03
BR-05	Only Admin manages slots, users, fee rules	Security Middleware	4.3 role matrix	TC-AUTH-05, TC-REP-01
BR-06	Completed records cannot be edited or deleted	Parking Transaction Service	4.1 state model	Not yet in STP

Coverage: 61 requirement items (33 FR, 12 NFR, 10 SEC, 6 BR) are all mapped to a component and a design section; 46 already have an STP v1.1 test case and 15 are marked "Not yet in STP".

3.9 Security Architecture

Security objectives (SRS 6.3): SO-1 Confidentiality of user, vehicle and parking data; SO-2 Integrity of slot status, parking records and fee rules; SO-3 Access control – privileged operations only for the right role; SO-4 Accountability – security-relevant events are recorded.

Security controls by layer:

- **Transport (SEC-04):** all client–server traffic over HTTPS with TLS 1.2 or higher, terminated at Nginx; plain HTTP is redirected to HTTPS; MongoDB connections use TLS and authentication.
- **Authentication (SEC-01, SEC-03, SEC-08):** passwords stored only as salted bcrypt hashes (cost 12); login checks lock-out first, then the password; 5 consecutive failures lock the account for 15 minutes.
- **Sessions (FR-03, SEC-07):** the JWT carries a session id (jti). Each request checks the session record: if it is revoked (logout) or idle for more than 15 minutes the request gets 401; otherwise lastActivityAt is updated. The token is kept in memory in the SPA, not in localStorage.
- **Authorisation (FR-06, SEC-02, SEC-09, BR-05):** the role matrix in Section 4.3 is enforced on the server for every endpoint – hiding a button in the UI is never relied on. For Vehicle Owners, every vehicle or record query is filtered by ownerId.
- **Input validation (SEC-05):** Joi schemas check type, length and format of every field; express-mongo-sanitize rejects keys starting with \$ or containing dots; text containing script content is rejected; React escapes output.
- **Accountability (SEC-06):** the Audit Logging Component records every failed login and every 401/403 with timestamp, username and source IP, without passwords, kept at least 90 days.
- **Secrets (SEC-10):** database URI and token-signing secret are read from environment variables (.env, not committed to Git); the server refuses to start without them.
- **Error responses (SRS 3.3):** no stack traces, database details or other users' data in any error.

Threat modelling (STRIDE):

Threat	Example in VPS	Mitigation
Spoofing	Guessing an attendant's password	bcrypt, lock-out after 5 failures (SEC-08), generic error (FR-02), audit (SEC-06)
Tampering	Sending a lower fee or a different exit time from the browser	Fee and timestamps computed on the server from stored rules and server time (FR-21, FR-22); input validation (SEC-05)
Repudiation	Attendant denies recording an entry or exit	recordedBy and server timestamps on every parking record; completed records cannot be changed (BR-06); audit log
Information disclosure	Owner looks up someone else's vehicle	Ownership filter (SEC-09), TLS (SEC-04), minimal error messages
Denial of service	Flooding login or search	Lock-out, request size limits, Nginx connection limits, indexed queries
Elevation of privilege	Attendant calls POST /slots or PUT /fee-rates	Server-side RBAC returns 403 (FR-06, SEC-02); role read only from the verified session, never

Threat	Example in VPS	Mitigation
		from the request body

3.10 Deployment View

Vehicle Parking System - Deployment View

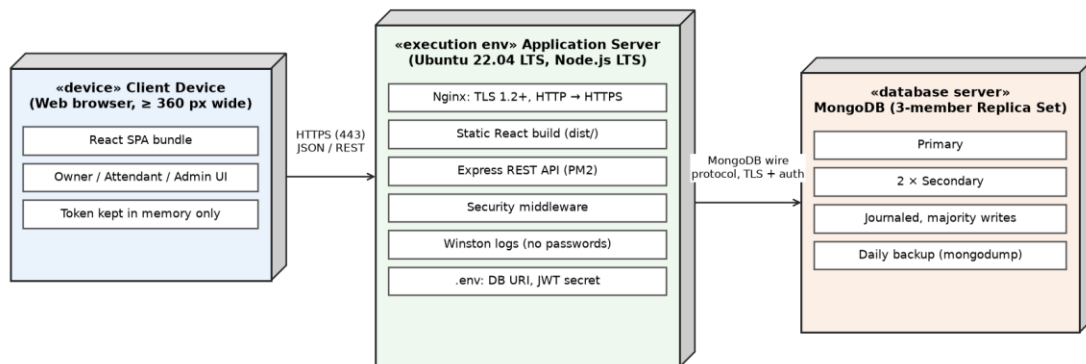


Figure 2: Deployment view of the Vehicle Parking System

Nginx serves the React build and forwards `/api/v1` requests to the Express API, which runs under PM2 (restarted automatically on failure). MongoDB runs as a 3-member replica set so that multi-document transactions are available and a single node failure does not stop the service (NFR-06, NFR-07). Each team member develops locally with a single-node replica set; tests use a separate test database (STP Section 6).

3.11 Architecture Decision Records (ADRs)

ADR	Decision	Status / Consequence
ADR-01	Layered MERN architecture with separate service modules	Accepted – satisfies NFR-10 and NFR-11; simple to deploy and test
ADR-02	Entry and exit-confirm run in MongoDB transactions, backed by partial-unique indexes on Active records	Accepted – guarantees NFR-05, BR-01, BR-03 even if application logic has a bug; needs a replica set
ADR-03	JWT plus server-side session record	Accepted – lets logout invalidate the token (FR-03) and enforces the 15-minute idle timeout (SEC-07)
ADR-04	Fees always computed on the server from stored rules at exit time	Accepted – prevents tampering; rate changes need no code change (FR-25 – FR-27, BR-04)
ADR-05	Slot view refreshes on demand and every 30 s instead of WebSockets	Accepted – no real-time requirement in the SRS; fewer moving parts

4. Design

4.1 Design Overview

The back end is organised by module (NFR-10). Every module follows the same layering:

```
server/src/  
  routes/      auth, users, vehicles, slots, parking,  
               fee-rates, reports (*.routes.js)  
  middleware/  authenticate (JWT + session), authorize(roles),  
               ownership, validate(schema), errorHandler  
  controllers/ thin: parse request -> call service -> HTTP response  
  services/    AuthService, UserService, VehicleService,  
               SlotService, ParkingService, FeeService,  
               ReportService, AuditService  
  models/      User, Session, Vehicle, Slot, ParkingRecord,  
               FeeRule, FeeSettings, AuditLog, Counter (Mongoose)  
  config/      env.js (reads .env, fails fast if missing)  
client/src/  
  pages/owner, pages/attendant, pages/admin, components/,  
  api/ (Axios client), context/AuthContext
```

Design rules: controllers contain no business logic; services depend on repositories that can be mocked in unit tests; all times come from the server clock; money is stored as integer paise.

State models:

- **Slot (FR-11 – FR-13, FR-19, FR-24):** AVAILABLE → OCCUPIED (entry) → AVAILABLE (exit confirmed); AVAILABLE ↔ UNAVAILABLE (Administrator). An OCCUPIED slot cannot be set UNAVAILABLE or change type.
- **Parking record (FR-19 – FR-24, BR-06):** ACTIVE (created at entry; exitTime and fee set when exit is recorded) → COMPLETED (exit confirmed). If the attendant cancels before confirming, exitTime and fee are cleared and the record stays ACTIVE. COMPLETED records are read-only.
- **User account (FR-05, SEC-08):** active ↔ deactivated (Administrator); locked for 15 minutes after 5 consecutive failed logins.

4.2 UML Sequence Diagrams

Four sequence diagrams cover the main flows: vehicle entry with slot allocation (SD-1), vehicle exit with fee calculation (SD-2), login (SD-3) and vehicle search/tracking (SD-4).

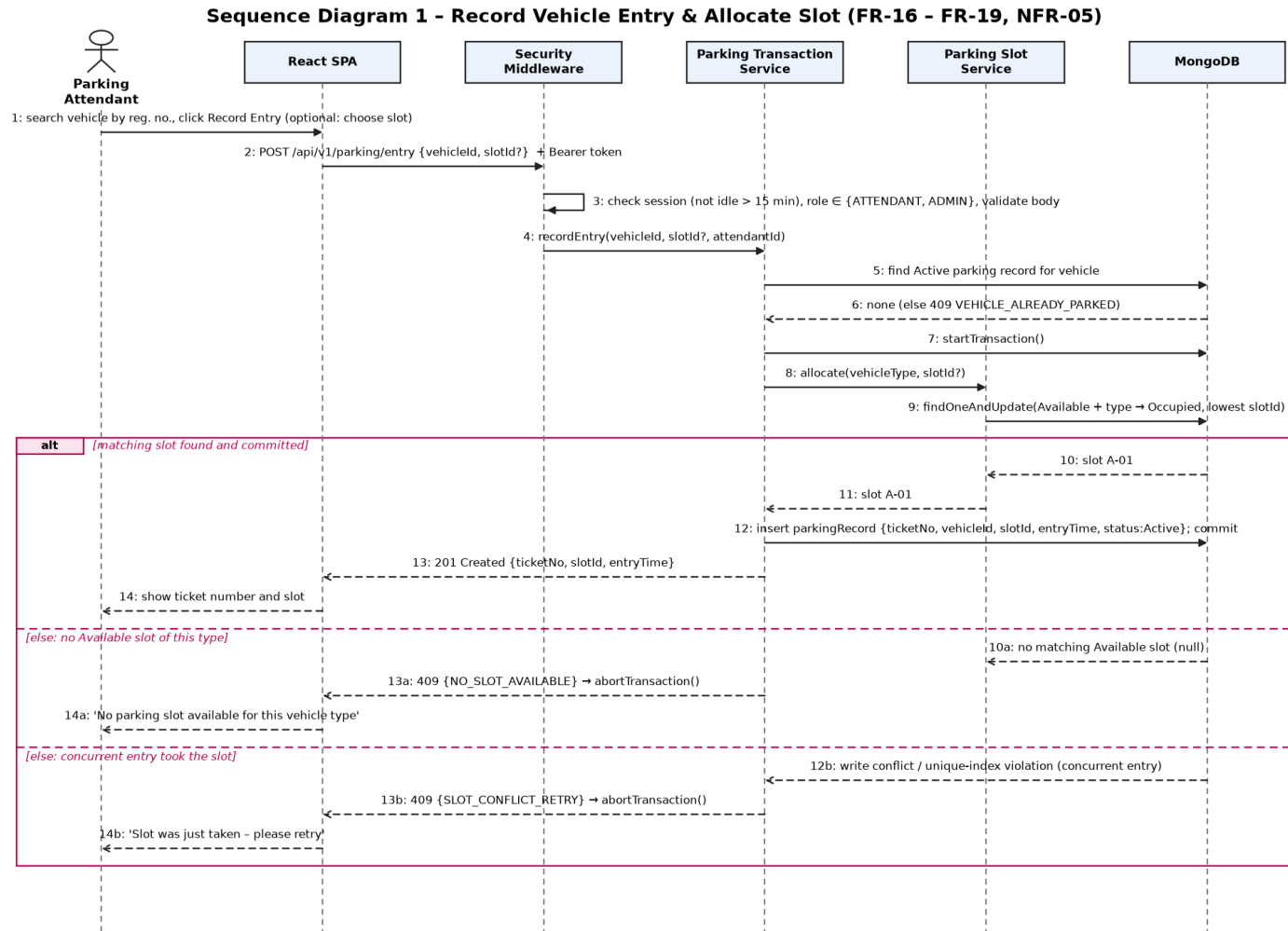


Figure 3: SD-1 – record vehicle entry and allocate a slot

SD-1: the attendant records entry for a registered vehicle. The Parking Transaction Service rejects the entry if the vehicle already has an Active record (FR-16), then, inside one transaction, the Parking Slot Service atomically switches the lowest-numbered Available slot of the right type to Occupied (or the slot the attendant chose, FR-17) and a parking record with a unique ticket number is inserted (FR-19). If no slot matches, the entry is refused (FR-18). If a concurrent entry wins the same slot, the transaction aborts and the attendant is asked to retry (NFR-05).

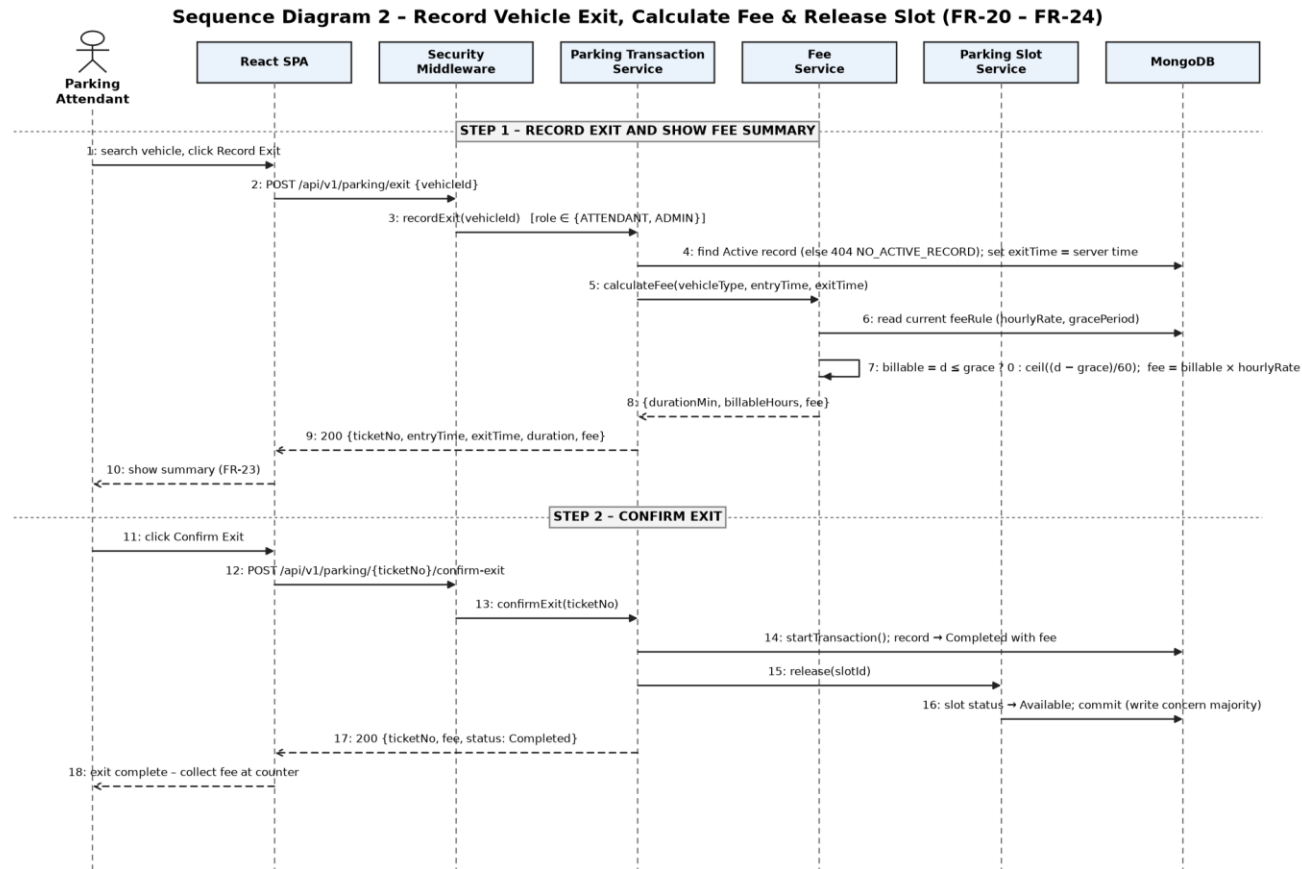


Figure 4: SD-2 – record vehicle exit, calculate fee and release the slot

SD-2: step 1 stores the exit time and computes the fee from the current fee rule, and the summary is shown before anything is final (FR-21 – FR-23). Step 2, on confirmation, marks the record Completed with the fee and sets the slot Available in one transaction (FR-24).

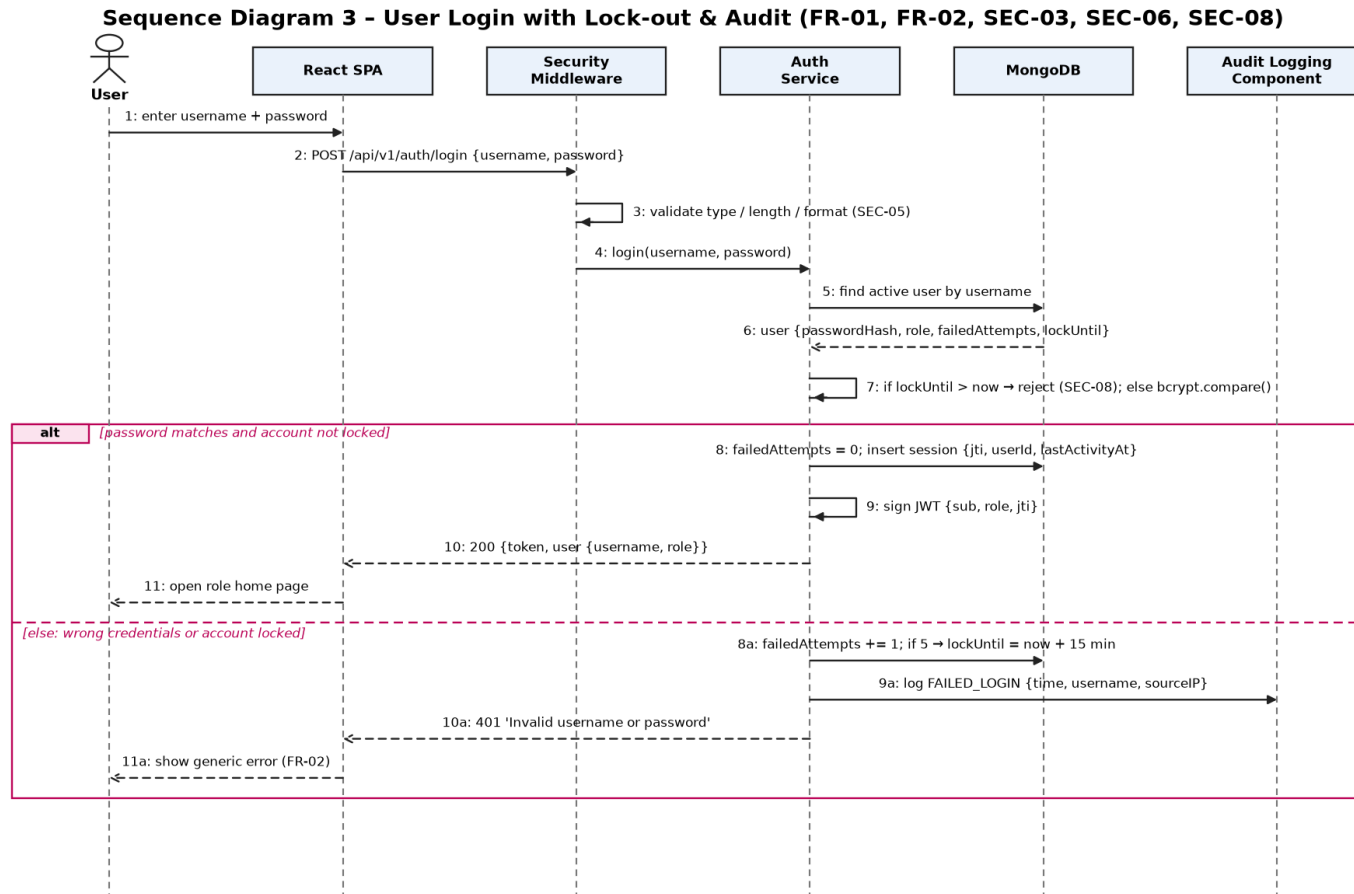


Figure 5: SD-3 – user login with lock-out and audit

SD-3: input is validated, a locked account is refused, and the password is checked with bcrypt. On success a server-side session is created and a JWT carrying its id is returned. On failure the attempt counter is increased (lock after 5), the failure is written to the audit log with username and source IP, and the generic message "Invalid username or password" is returned.

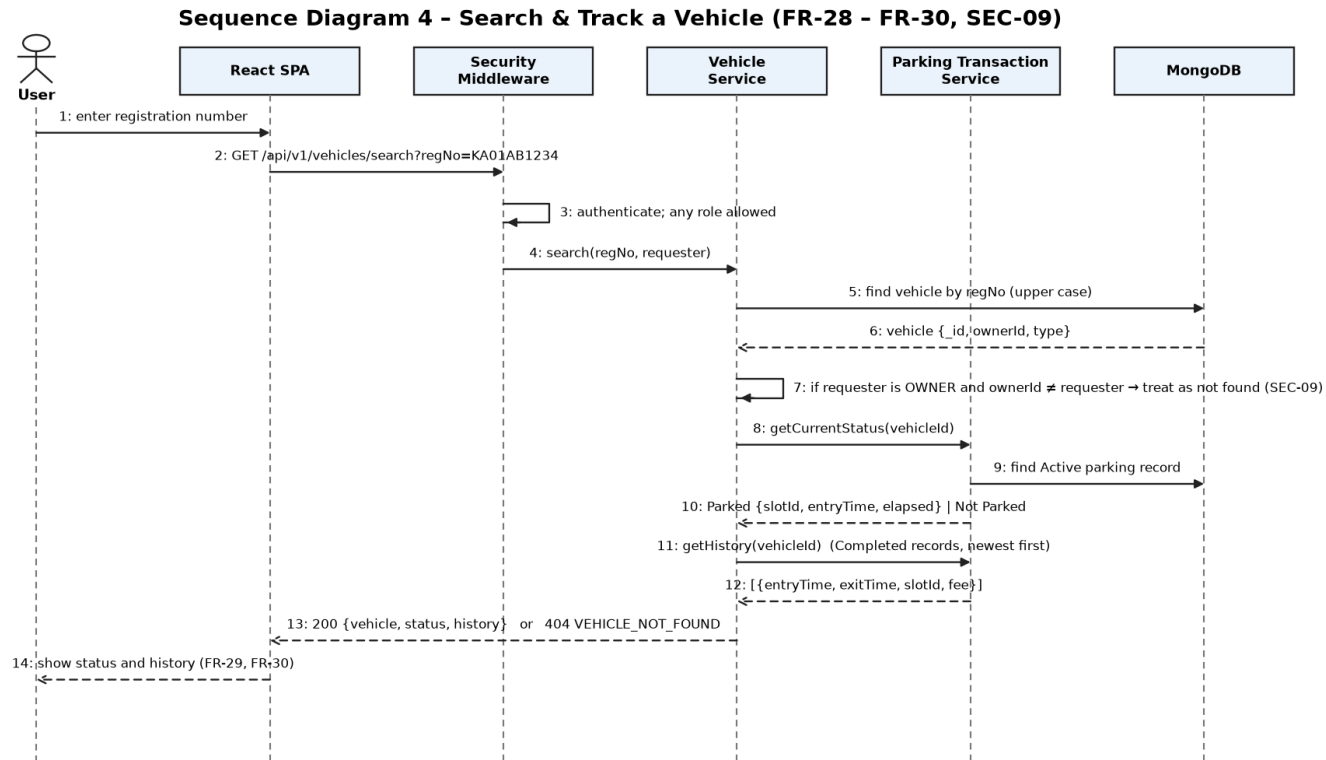


Figure 6: SD-4 – search and track a vehicle

SD-4: any logged-in user can search by registration number. For a Vehicle Owner, vehicles registered to someone else are reported as not found (SEC-09). The response gives the current status – Parked with slot, entry time and elapsed time, or Not Parked – and the history of Completed records, newest first.

4.3 API Design

Base URL `https://<host>/api/v1`. JSON requests and responses; protected endpoints need the header `Authorization: Bearer <token>`. Status codes follow SRS 3.3 (200, 201, 400, 401, 403, 404, 409, 500).

Endpoint and role matrix (enforced on the server; used by STP TC-SEC-02). Y = allowed, Own = only own vehicles/records, – = 403 Forbidden.

Component	Endpoint	Method	Public	Owner	Attend.	Admin	Req.
Auth	/auth/register	POST	Y	–	–	–	FR-04
Auth	/auth/login	POST	Y	Y	Y	Y	FR-01, FR-02
Auth	/auth/logout	POST	–	Y	Y	Y	FR-03
User Mgmt	/users	GET, POST	–	–	–	Y	FR-05
User Mgmt	/users/{userId}	PUT	–	–	–	Y	FR-05
User Mgmt	/users/{userId}/status	PATCH	–	–	–	Y	FR-05
Vehicle	/vehicles	POST	–	Own	Y	Y	FR-07 – FR-09
Vehicle	/vehicles/{vehicleId}	PUT	–	Own	–	Y	FR-10
Vehicle	/vehicles/{vehicleId}/deactivate	PATCH	–	Own	–	Y	FR-10
Vehicle	/vehicles/search?regNo=	GET	–	Own	Y	Y	FR-28, FR-29
Parking	/vehicles/{vehicleId}/history	GET	–	Own	Y	Y	FR-30
Slot	/slots, /slots/summary	GET	Y	Y	Y	Y	FR-14, FR-15, SEC-01
Slot	/slots	POST	–	–	–	Y	FR-11
Slot	/slots/{slotId}	PUT	–	–	–	Y	FR-12
Slot	/slots/{slotId}/status	PATCH	–	–	–	Y	FR-13
Parking	/parking/entry	POST	–	–	Y	Y	FR-16 – FR-19
Parking	/parking/exit	POST	–	–	Y	Y	FR-20 – FR-23
Parking	/parking/{ticketNo}/confirm-exit	POST	–	–	Y	Y	FR-24
Parking	/parking/{ticketNo}/cancel-exit	POST	–	–	Y	Y	FR-23
Parking	/parking/active	GET	–	–	Y	Y	FR-31
Fee	/fee-rates	GET	–	–	Y	Y	FR-25, FR-26
Fee	/fee-rates	PUT	–	–	–	Y	FR-25 – FR-27
Report	/reports/occupancy	GET	–	–	–	Y	FR-32
Report	/reports/vehicle-tracking	GET	–	–	–	Y	FR-33

Slot availability is readable without login because SEC-01 exempts it; all other functions need authentication.

Detailed interface definitions:

(a) Auth Service – Login

Endpoint: `/api/v1/auth/login` Method: `POST` Role: `Public`
Request: `{ "username": "att1", "password": "*****" }`
Response: `200 OK`
`{ "token": "<JWT with jti>",`
`"user": { "username": "att1", "role": "ATTENDANT" } }`
Errors: `400 VALIDATION_ERROR` (missing / over-long / wrong type)
`401 INVALID_CREDENTIALS` "Invalid username or password"
`401 ACCOUNT_LOCKED` (5 failures; try again after 15 min)

(b) Vehicle Service – Register Vehicle

Endpoint: `/api/v1/vehicles` Method: `POST`
Role: `OWNER` (owner = self), `ATTENDANT`, `ADMIN`
Request: `{ "regNo": "ka01ab1234", "vehicleType": "CAR" }`

Response: 201 Created
 { "vehicleId": "6703cd...", "regNo": "KA01AB1234",
 "vehicleType": "CAR", "ownerId": "66f1c0..." }
Errors: 400 VALIDATION_ERROR { "field": "vehicleType" } (empty /
 not TWO_WHEELER | CAR | OTHER; regNo not 4-15 chars of
 letters, digits, hyphen, space)
 409 DUPLICATE_REG_NO { "field": "regNo" }

(c) Parking Slot Service – Add Slot / Slot Summary

Endpoint: /api/v1/slots Method: POST Role: ADMIN
Request: { "slotId": "A-01", "slotType": "CAR" }
Response: 201 Created { "slotId": "A-01", "slotType": "CAR",
 "status": "AVAILABLE" }
Errors: 400 VALIDATION_ERROR 409 DUPLICATE_SLOT_ID 403 FORBIDDEN

Endpoint: /api/v1/slots/summary Method: GET Role: Public
Response: 200 OK { "total": 17, "byType": [
 { "slotType": "CAR", "total": 10, "available": 6,
 "occupied": 3, "unavailable": 1 }, ...] }

(d) Parking Transaction Service – Record Entry

Endpoint: /api/v1/parking/entry Method: POST
Role: ATTENDANT, ADMIN
Request: { "vehicleId": "6703cd...", "slotId": "A-05" } (slotId optional)
Response: 201 Created
 { "ticketNo": "T-20261007-0042", "slotId": "A-01",
 "entryTime": "2026-10-07T09:04:12+05:30" }
Errors: 404 VEHICLE_NOT_FOUND
 409 VEHICLE_ALREADY_PARKED
 409 NO_SLOT_AVAILABLE "No parking slot available for this
 vehicle type"
 409 SLOT_NOT_AVAILABLE (chosen slot busy or wrong type)
 409 SLOT_CONFLICT_RETRY (concurrent entry; please retry)

(e) Parking Transaction Service – Record Exit and Confirm

Endpoint: /api/v1/parking/exit Method: POST
Role: ATTENDANT, ADMIN
Request: { "vehicleId": "6703cd..." }
Response: 200 OK
 { "ticketNo": "T-20261007-0042", "slotId": "A-01",
 "entryTime": "2026-10-07T09:04:12+05:30",
 "exitTime": "2026-10-07T10:34:12+05:30",
 "durationMin": 90, "billableHours": 2,
 "hourlyRate": "30.00", "fee": "60.00" }
Errors: 404 NO_ACTIVE_RECORD

Endpoint: /api/v1/parking/{ticketNo}/confirm-exit Method: POST
Response: 200 OK { "ticketNo": "...", "status": "COMPLETED",
 "fee": "60.00", "slotStatus": "AVAILABLE" }

Errors: 404 TICKET_NOT_FOUND 409 ALREADY_COMPLETED

(f) Fee Service – Update Fee Rules

Endpoint: /api/v1/fee-rates Method: PUT Role: ADMIN
Request: { "rates": [{ "vehicleType": "TWO_WHEELER", "hourlyRate": 10 },
{ "vehicleType": "CAR", "hourlyRate": 30 },
{ "vehicleType": "OTHER", "hourlyRate": 50 }],
"gracePeriodMin": 15 }
Response: 200 OK (stored rules; used for exits recorded from now on)
Errors: 400 VALIDATION_ERROR (rate < 0 or not a number;
grace not a whole number 0-60)
403 FORBIDDEN

(g) Vehicle Service – Search and Track

Endpoint: /api/v1/vehicles/search?regNo=KA01AB1234 Method: GET
Role: OWNER (own vehicles only), ATTENDANT, ADMIN
Response: 200 OK
{ "regNo": "KA01AB1234", "vehicleType": "CAR",
"status": "PARKED", "slotId": "A-01",
"entryTime": "2026-10-07T09:04:12+05:30",
"elapsedMin": 42 }
or { ..., "status": "NOT_PARKED" }
Errors: 404 VEHICLE_NOT_FOUND (also for another owner's vehicle)

4.4 Error Handling, Logging & Monitoring

Error handling:

- Services throw typed errors (code, HTTP status, message); one Express error handler turns them into the standard body: { "error": { "code": "NO_SLOT_AVAILABLE", "message": "No parking slot available for this vehicle type", "field": null } }.
- Every message says what is wrong and what the user can do; validation errors name the offending field (FR-08, NFR-09).
- Unexpected errors return 500 with a generic message; stack traces and database details are never sent to the client (SRS 3.3).
- Entry and exit-confirm run in transactions and are rolled back on any failure, so slot status and records stay consistent.
- Front end: the Axios client shows the message next to the field or as a banner; a 401 sends the user to the Login page.

Logging:

- Winston JSON application logs with time, request id, user, route, status and duration; passwords and tokens are never logged (SEC-03).
- Audit log (SEC-06): failed logins and all 401/403 responses with timestamp, username and source IP, kept at least 90 days.

Monitoring:

- GET /api/v1/health reports API and database status; PM2 restarts the API on crash.
- Response-time logging per route gives the 95th-percentile figures needed for NFR-01 – NFR-04; uptime is tracked against operating hours for NFR-07.
- Indexes on regNo, slotId, slot type/status and Active records keep search, slot display, entry and exit fast (NFR-01 – NFR-04).

4.5 UX Design

- **Screens (SRS 3.1):** Login/Register, Slot Availability, Vehicle Registration and Search, Vehicle Entry, Vehicle Exit with fee summary, Parking History, Slot Management, User Management, Fee Rules, Reports. Menus show only the screens allowed for the role.
- **Every screen:** logged-in username, role and a Logout control; mandatory fields marked with *; field-level error messages.
- **Slot status:** text plus colour – Available (green), Occupied (red), Unavailable (grey) – with counts per slot type at the top.
- **Fast entry (NFR-08):** type registration number → Search → Record Entry → Confirm: 4 actions; the suggested slot is pre-selected.
- **Exit (FR-23):** the summary shows entry time, exit time, duration and fee before Confirm Exit; Cancel leaves the record Active.
- **Responsive (NFR-12):** layout works from 360 px wide on the latest two versions of Chrome, Firefox and Edge.

4.6 Open Issues & Next Steps

- Default hourly rates and grace period (SRS 2.6) to be confirmed by the reviewer before system testing.
- Align the Test Plan with SRS v1.1: STP v1.1 uses older requirement numbers (FR-01 – FR-23, NFR-01 – NFR-07, SEC-01 – SEC-07), treats the grace period as fixed, and excludes Vehicle Owner functions that the SRS includes. Add test cases for the items marked "Not yet in STP" in Section 3.8.
- STP TC-SEC-01 expects the Slot Availability page to need login, but SEC-01 allows it without login; the team should decide which is intended.
- Fill in the Code File Reference column of the RTM as implementation progresses.
- Possible later versions: number-plate recognition, online payment, reservations, mobile app (all out of scope for Version 1.0).

5. Appendices

5.1 Glossary:

See Section 1.4 and SRS Appendix A (Vehicle, Parking Slot, Slot Status, Parking Record, Vehicle Tracking, Grace Period, Parking Fee, Vehicle Owner, Parking Attendant, Administrator).

5.2 References:

- Team 10 – Vehicle Parking System SRS v1.1 and Software Test Plan v1.1.
- PES University, SE (UE24CS341A) Mini-Project Problem Statements – Section F; Mini-Project Deliverables Part-1; Project Guidelines; SAD template.
- ISO/IEC/IEEE 42010:2022 – Architecture description; IEEE Std 1016-2009 – Software Design Descriptions.
- ISO/IEC/IEEE 29148:2018 – Requirements engineering; IEEE Std 829-2008 – Software test documentation.
- OWASP Top 10 and OWASP Application Security Verification Standard; NIST SP 800-160.

5.3 Tools:

Python/matplotlib (UML diagrams in docs/diagrams), draw.io / PlantUML, Swagger / OpenAPI 3.0, Postman, VS Code, Git & GitHub, MongoDB Compass, Jest, JMeter.